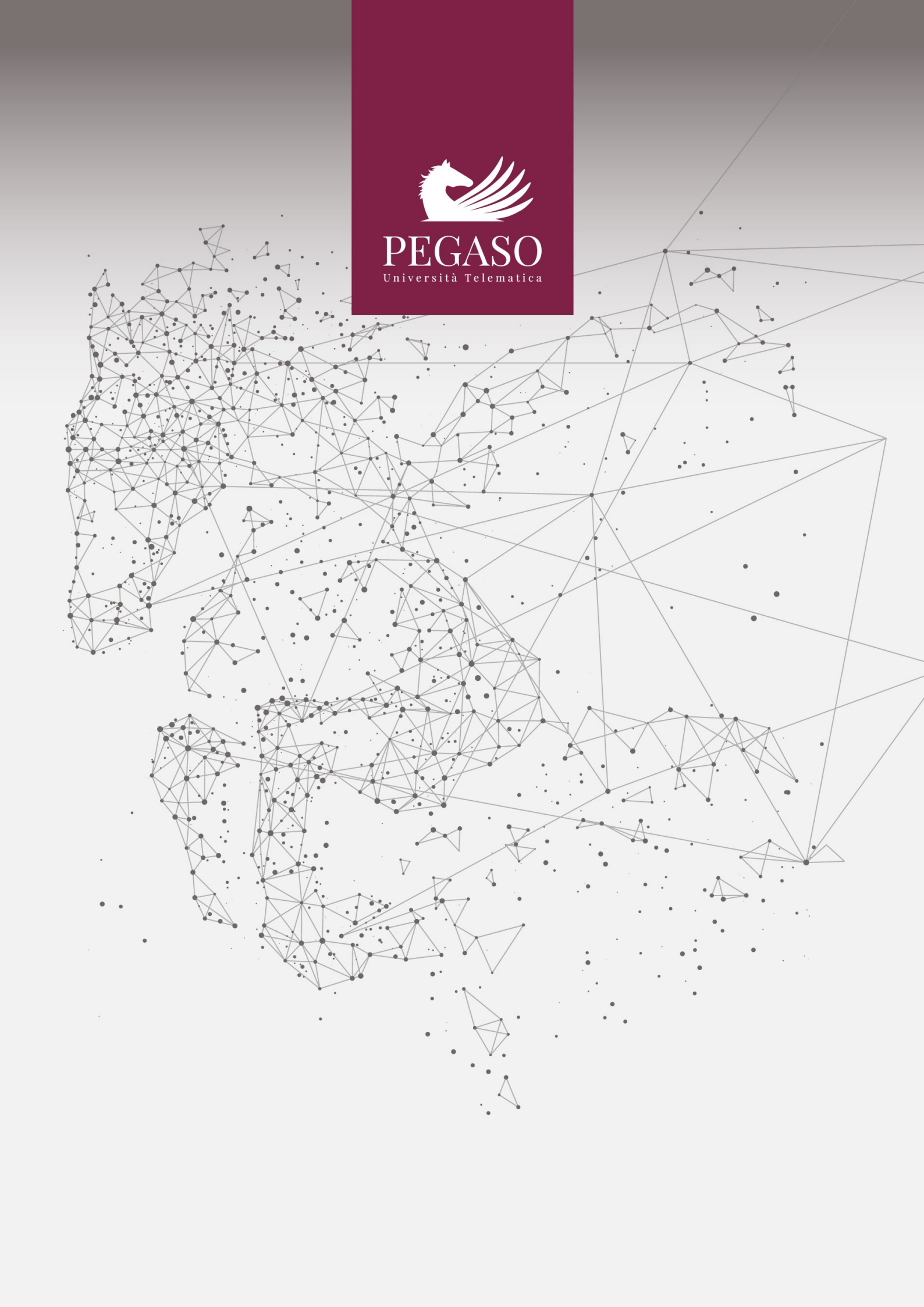

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. OPERAZIONI DI MODIFICA E VERIFICA DEGLI ARRAY....4
3. TRATTARE GLI ARRAY COME INSIEMI....11
4. LAVORARE CON I FILE....13
5. ESERCIZIO SULLA VARIAZIONE DI VALORI....16
BIBLIOGRAFIA....20

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano alcune operazioni avanzate di modifica e di verifica degli array quali lo slicing su matrici, l'inserimento di valori negli array, la definizione e l'applicazione dei filtri agli array, le funzioni condizionali e l'ordinamento degli array. Si presentano, poi, le funzioni e le potenzialità offerte da NumPy per trattare gli array come insiemi con tutte le proprietà caratteristiche degli insiemi matematici. Si illustrano anche le modalità per salvare e caricare gli array da file di testo e da file binari al fine di memorizzare in modo permanente i risultati delle elaborazioni.

Infine, si presenta un esercizio sul calcolo delle variazioni di titoli di borsa che mostra tutte le potenzialità offerte da NumPy in codice Python che si presenta molto compatto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni di modifica e verifica degli array

Lo **slicing** sugli array a più dimensioni permette di ottenere delle sottomatrici. Nel caso illustrativo riportato si eseguono le seguenti operazioni:

- si definisce un array di 28 elementi ;
- si modifica la forma attraverso la funzione reshape() riorganizzandolo in una matrice 7x4;
- si estrae un elemento di coordinate (1,2) nel seguente modo m[1][2];
- si estrae una riga (la seconda) della matrice nel seguente modo m[1];
- si estrae la sottomatrice composta dalle ultime quattro righe nel seguente modo m[3:6];
- si estrae la seconda riga della sottomatrice nel seguente modo m[3:6][1];
- si estrae il secondo valore della seconda riga della sottomatrice nel seguente modo m[3:6][1][1].

```
import numpy as np
m = np.array(range(1,29)).reshape(7,4)
print(m)
print("Elemento alle coordinate (1,2) di m = ",m[1][2])
print("Seconda riga di m", m[1])
print("Sottomatrice ")
print(m[3:6])
print("Seconda riga della sottomatrice", m[3:6][1])
print("Secondo valore della seconda riga della sottomatrice", m[3:6][1][1])
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]
 [25 26 27 28]]
Elemento alle coordinate (1,2) di m =  7
Seconda riga di m [5 6 7 8]
Sottomatrice
[[13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
Seconda riga della sottomatrice [17 18 19 20]
Secondo valore della seconda riga della sottomatrice 18
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

NumPy offre anche altre funzioni per **automatizzare le operazioni di inserimento** come:

- **append**() per l'accodamento di un valore alla struttura dati;
- **insert** () per il posizionamento di un valore in un punto specifico della struttura dati.

Ad ogni modo bisogna considerare due aspetti:

- le funzioni append() e insert() non vengono invocate sugli oggetti array, ma mediante il modulo numpy importato;
- tali funzioni non modificano l'oggetto su cui agiscono, ma ne restituiscono una nuova versione che include il valore inserito.

Dopo l'invocazione di append() viene restituito un array, ma non appena se ne verifica il contenuto, è possibile osservare che l'array di partenza non ha subito modifiche.

```
import numpy as np
v = np.array(range(1,10))
print("v =",v)
print("Append",np.append(v,100))
print("v=",v)
```

```
v = [1 2 3 4 5 6 7 8 9]
Append [  1   2   3   4   5   6   7   8   9 100]
v= [1 2 3 4 5 6 7 8 9]
```

Stessa cosa accade con la funzione insert(). Si definisce un nuovo valore da inserire in posizione 4, ma al momento della stampa del vettore non ci sono state modifiche come si può osservare dal codice seguente.

```
import numpy as np
v = np.array(range(1,10))
print("v =",v)
print("Insert ",np.insert(v,3,555))
print("v =",v)
```

```
v = [1 2 3 4 5 6 7 8 9]
Insert  [  1   2   3 555   4   5   6   7   8   9]
v = [1 2 3 4 5 6 7 8 9]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una volta inizializzati gli array, una delle operazioni più importanti da saper compiere è **l'applicazione di filtri** che permettano di scegliere una **selezione dei dati in base a determinati criteri.** Il risultato di un filtro su un array sarà un array a sua volta.

La metodologia da applicare per filtrare i dati è molto sintetica e pratica poiché si basa **sull'inserimento di confronti tra parentesi quadre.** Per esempio, su un array contenente numeri interi sia positivi che negativi vogliamo applicare un filtro per ottenere un **array di soli valori negativi.**

Si sottolinea che i due array saranno disgiunti, quindi le modifiche su uno dei due vettori non avranno alcun riflesso sull'altro. Attraverso l'utilizzo della forma compatta, il secondo array sarà pari a v[v<0].

```
import numpy as np
v = np.array([12, 21, 54, 78, 9, -12, 14, -6, 50, -31])
print("v=",v)
v2 = v[v<0]
print("v2=",v2)
```

```
v= [ 12  21  54  78   9 -12  14  -6  50 -31]
v2= [-12  -6 -31]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Altri esempi di filtraggio dei valori dell'array riportati di seguito sono i seguenti:

- array con tutti i valori maggiori di 9;
- array con tutti i valori maggiori o uguali a 9;
- array con tutti i valori uguali a 9;
- array con tutti i valori pari (resto della divisione intera con due pari a zero);
- array con tutti i valori diversi da 78;
- array con tutti i valori minori di 0 o maggiori di 50;
- array con tutti i valori inclusi nell'intervallo (0,50) estremi inclusi.

```
import numpy as np
v = np.array([12, 21, 54, 78, 9, -12, 14, -6, 50, -31])
print("v=",v)
print("Maggiore di 9\t",v[v>9])
print("Maggiore o uguale a 9\t",v[v>=9])
print("Uguale a 9\t",v[v==9])
print("Pari\t",v[v%2==0])
print("Diverso da 78\t",v[v!=78])
print("Minore di 0 o maggiore di 50\t",v[ (v<0) | (v>50)])
print("Tra 0 e 50 estremi inclusi\t",v[ (v>=0) & (v<=50)])
```

```
v= [ 12  21  54  78   9 -12  14  -6  50 -31]
Maggiore di 9    [12 21 54 78 14 50]
Maggiore o uguale a 9    [12 21 54 78  9 14 50]
Uguale a 9       [9]
Pari     [ 12  54  78 -12  14  -6  50]
Diverso da 78    [ 12  21  54   9 -12  14  -6  50 -31]
Minore di 0 o maggiore di 50     [ 54  78 -12  -6 -31]
Tra 0 e 50 estremi inclusi       [12 21  9 14 50]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I confronti dei valori per il filtraggio dei dati possono essere eseguiti anche con le **funzioni condizionali.**

La funzione **where(c,a,b)** è la versione NumPy dell'operatore ternario if-else ed opera su array aventi la stessa forma (shape). La funzione where() richiede un array booleano (c) e altri due array (a e b) e restituisce:

- l'array d[i] = a[i] se c[i]==True;
- altrimenti d[i] = b[i].

Un esempio di applicazione della funzione where() potrebbe essere quella di sostituire in un vettore tutti i numeri positivi con il simbolo + e tutti quelli negativi con il simbolo – come nel caso che segue:

```
import numpy as np
v = np.array([12, 21, 54, 78, 9, -12, 14, -6, 50, -31])
v2 = np.where(v<0,'-','+')
print(v2)
```

```
['+' '+' '+' '+' '+' '-' '+' '-' '+' '-']
```

La funzione where() restituisce anche le posizioni in cui ha trovato gli elementi cercati. Si supponga di voler cercare nel vettore dei primi 9 numeri la posizione del numero 5. Sia r il risultato di np.where(v==5). r[0] conterrà l'array di tutte le posizioni dell'array di partenza che verificano la condizione, mentre r[0][0] rappresenta la posizione della prima occorrenza dell'elemento cercato.

```
import numpy as np
v = np.array(range(1,10))
print(v)
r = np.where(v==5)
print(r)
print(r[0])
print(r[0][0])
```

```
[1 2 3 4 5 6 7 8 9]
(array([4], dtype=int64),)
[4]
4
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel nostro caso, il risultato è una tupla (array([4], dtype=int64)) costituita da un solo elemento. Nel caso di più elementi trovati attraverso la funzione where() si osserva che r[0] conterrà tutte le posizioni dell'array di partenza i cui elementi verificano la condizione (condizione nell'esempio >=5). Il numero delle posizioni trovate sarà pari a len(r[0]).

```
import numpy as np
v = np.array(range(1,10))
print(v)
r = np.where(v>=5)
print(r)
print(r[0])
print("Len del risultato", len(r[0]))
for i in range(len(r[0])):
    print(r[0][i])
```

```
[1 2 3 4 5 6 7 8 9]
(array([4, 5, 6, 7, 8], dtype=int64),)
[4 5 6 7 8]
Len del risultato 5
4
5
6
7
8
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

NumPy offre anche le funzioni condizionali **any**() e **all**() che restituiscono True se, rispettivamente, uno qualsiasi o tutti gli elementi di un array sono True (se array numerico 0 = False, numero diverso da 0 = True). La funzione **nonzero**(), invece, restituisce gli indici di tutti gli elementi diversi da zero.

Nell'esempio che segue, si inizializza un array di dimensione 11 con gli elementi consecutivi da -5 a 5. Si richiama la funzione nonzero() che restituirà un array contenente 10 posizioni in cui mancherà solo la posizione di indice 5 in cui c'è l'elemento 0.

La funzione any() restituirà True perché ci sono 10 elementi diversi da zero, mentre all() restituirà False perché c'è almeno uno 0.

```
import numpy as np
v = np.array(range(-5,6))
print(v)
print("Non zero", np.nonzero(v))
v2 = v>0
print("Any",np.any(v2))
print("All",np.all(v2))
```

```
[-5 -4 -3 -2 -1  0  1  2  3  4  5]
Non zero (array([ 0,  1,  2,  3,  4,  6,  7,  8,  9, 10], dtype=int64),)
Any True
All False
```

L'ordinamento dei dati è possibile con la funzione **sort**() di numpy, come mostrato nel caso seguente di 5 numeri casuali nel range 0:100.

```
import numpy as np
v_casuale = np.random.random((5))*100
print(v_casuale)
print("Array ordinato: ",np.sort(v_casuale))
```

```
[55.81104805 79.31605668 82.20367199 41.98661862 38.88739314]
Array ordinato:  [38.88739314 41.98661862 55.81104805 79.31605668 82.20367199]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Trattare gli array come insiemi

In alcuni casi, l'ordine degli elementi di un array è meno importante della composizione dell'array stesso. A volte, infatti, è più importante sapere se un determinato elemento è presente o meno nell'array o che tipi di elementi sono presenti nell'array.

NumPy è in grado anche di trattare gli array come insiemi matematici. La funzione **unique**(x) restituisce un array di tutti gli elementi univoci (cioè non ripetuti) di x.

Esercizio sugli array trattati come insiemi: sappiamo che la bioinformatica è una scienza di grande interesse che si occupa di sequenze di geni, valutando l'ordine dei nucleotidi nel DNA. Proviamo a svolgere alcune operazioni di pseudo-bioinformatica e vediamo quanti tipi di nucleotidi esistono in un determinato frammento di DNA. Partiremo da una stringa di DNA che trasformeremo prima in lista e poi in array. Richiamando la funzione unique() sull'array otterremo l'array nucleotidi formato da 'A', 'C', 'G' E 'T'.

La funzione **in1d**(a,b) restituisce un array booleano dove un elemento è True se l'elemento corrispondente di a è in b.

```
import numpy as np
dna = "AGTCCGCGAATACAGGCTCGGT"
dna_array = np.array(list(dna))
print(dna_array)
nucleotidi = np.unique(dna_array)
print("Nucleotidi: ",nucleotidi)
```

```
['A' 'G' 'T' 'C' 'C' 'G' 'C' 'G' 'A' 'A' 'T' 'A' 'C' 'A' 'G' 'G' 'C' 'T'
 'C' 'G' 'G' 'T']
Nucleotidi:  ['A' 'C' 'G' 'T']
```

Gli array a e b non devono necessariamente avere la stessa forma come è possibile osservare nel caso seguente:

```
import numpy as np
v = np.array(range(10))
print(v)
v2 = np.array([3,5,9,11,13])
print(np.in1d(v2,v))
```

```
[0 1 2 3 4 5 6 7 8 9]
[ True  True  True False False]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le funzioni **union1d**() e **intersect1d**() calcolano l'unione e l'intersezione degli insiemi rappresentati da **due array monodimensionali.** Anche qui gli array non devono necessariamente avere le stesse dimensioni. Nel caso che segue, i due insiemi sono disgiunti (numeri pari e dispari) e l'intersezione sarà un insieme vuoto.

```
import numpy as np
d = np.array(range(1,10,2))
p = np.array(range(0,10,2))
print("Dispari",d)
print("Pari",p)
print("Unione",np.union1d(p,d))
print("Intersezione",np.intersect1d(p,d))
```

```
Dispari [1 3 5 7 9]
Pari [0 2 4 6 8]
Unione [0 1 2 3 4 5 6 7 8 9]
Intersezione []
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Lavorare con i file

Qualora fosse necessario salvare il contenuto di un array su un file oppure leggere dati da file e importarli velocemente in un array NumPy, ci sono due possibilità:

- **file di testo;**
- **file binari.**

Per **i file di testo**, il formato più diffuso è il CSV (Comma-Separated Values). Considerata l'efficacia dello strumento Pandas per i Big Data (che introdurremo nei prossimi moduli), ci limiteremo ad utilizzare file di testo con un solo valore per riga.

Prendiamo come esempio un file contenente un insieme di dati numerici di origine finanziaria (es. prezzi giornalieri di un titolo azionario). Attraverso la funzione **genfromtxt**() di NumPy è possibile leggere il file di testo che si chiama *prezzi.txt* che ha il seguente formato:

```
12.28
12.82
13.27
13.01
12.54
12.75
12.88
12.01
11.56
11.1
```

Nel codice sorgente che segue, i valori letti dal file prezzi.txt saranno trasformati in numeri in virgola mobile e inseriti in un array.

```
import numpy as np
prezzi = np.genfromtxt('prezzi.txt')
print(prezzi)
print("Numero prezzi letti", prezzi.size)
print("Tipo di dati letti", prezzi.dtype)
```

```
[12.28 12.82 13.27 13.01 12.54 12.75 12.88 12.01 11.56 11.1 ]
Numero prezzi letti 10
Tipo di dati letti float64
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Supponiamo ora di voler aumentare del 20% i numeri letti e di voler salvare su file di testo il nuovo array. Attraverso la funzione **savetxt**() di NumPy è possibile scrivere su file di testo il nuovo array. Nel codice sorgente che segue, con la funzione around() di NumPy si aumenta del 20% il valore di tutti gli elementi dell'array prezzi (prezzi*1,2) e si arrotonda alla seconda cifra decimale. Con la funzione savetxt(), si salva sul file *prezzi_new.txt* l'array *prezzi_au* con formato stabilito dalla stringa '%0.2f' (due cifre decimali).

```
import numpy as np
prezzi = np.genfromtxt('prezzi.txt')
prezzi_au = np.around(prezzi *1.2,2)
print(prezzi_au)
np.savetxt('prezzi_new.txt',prezzi_au, fmt='%0.2f')
```

```
[14.74 15.38 15.92 15.61 15.05 15.3  15.46 14.41 13.87 13.32]
```

Il file prezzi_new.txt conterrà i seguenti valori:

```
14.74
15.38
15.92
15.61
15.05
15.30
15.46
14.41
13.87
13.32
```

Per quanto riguarda il salvataggio su **file binario**, invece, è possibile salvare dati in file in formato .npy allo scopo di recuperare i dati e usarli sempre in NumPy. Per salvare su memoria di massa si utilizza la funzione **save**() come mostrato nel codice seguente ('dati_binari' è il nome del file su cui verrà salvato l'array prezzi).

```
import numpy as np
prezzi = np.array([12.28, 12.82, 13.27, 13.01, 12.54, 12.75, 12.88, 12.01, 11.56, 11.1])
np.save('dati_binari', prezzi)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Verrà immediatamente creato il file dati_binari con estensione npy con i valori dell'array.

Per leggere da file binario si utilizza la funzione **load**() passandole il nome del file inclusa l'estensione.

```
import numpy as np
prezzi = np.load('dati_binari.npy')
print(prezzi)
```

```
[12.28 12.82 13.27 13.01 12.54 12.75 12.88 12.01 11.56 11.1 ]
```

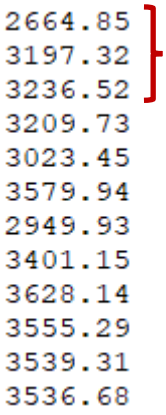
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Esercizio sulla variazione di valori

Supponiamo di avere su file un elenco di valori finanziari registrati in ordine cronologico crescente. Si avranno, quindi, alle prime righe i valori meno recenti e alle ultime righe i valori più recenti. Per semplicità si riporta solo il valore della chiusura di un titolo al termine della giornata.

L'esercizio[1] consiste nell'utilizzare le funzionalità di NumPy per individuare le maggiori variazioni riscontrate (in negativo e in positivo) nell'arco di un predeterminato numero di giorni. L'obiettivo a cui siamo interessati sono le variazioni intese come differenza tra i valori estremi dell'intervallo. Osservando l'estratto del file riportato di seguito, se l'intervallo per l'esecuzione è pari a 3 giorni, al terzo numero corrisponderà la variazione 571,67 come differenza tra 3.236,52 (terzo giorno) e 2.664,85 (primo giorno).



[1] Esercizio tratto dal libro "Data Science con Python", G. Maggi, Edizioni LSWR – pag. 136

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per risolvere l'esercizio si sceglie il seguente metodo:

1. si caricano i dati dal file nell'array;
2. si crea un altro array con i medesimi valori scalati di X posizioni (X = numero di giorni su cui calcolare la variazione);
3. si svolgono le operazioni di confronto tra i due array ottenendo un terzo array con i risultati;
4. si esprimono considerazioni sui risultati ottenuti.

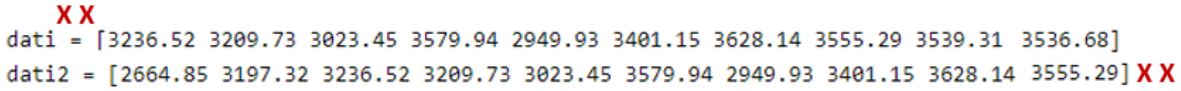
Procediamo scrivendo il codice delle funzioni principali:

1. **carica_dati**() per estrarre i dati dal file (verrà utilizzata la funzione genfromtxt());
2. **crea_array_confronto**() per generare una versione dell'array di partenza scalata di tante posizioni quanti sono i giorni dell'intervallo (se l'array si chiama *prezzi* e il numero di giorni *periodo*, l'estrazione sarà possibile nel seguente modo *prezzi[:-periodo+1]*);
3. **calcola_incrementi**() per produrre l'array dei risultati (se *dati2* è l'array di confronto e *dati* è l'array di partenza scalato nel seguente modo *dati[periodo-1:]*, le variazioni sono calcolabili come *dati2-dati*). Vedi figura successiva in cui lo scalamento è evidenziato dalle X in rosso;
4. **statistiche**() per visualizzare le stringhe che sintetizzano i risultati. Si andrà alla ricerca della variazione minima e massima e si individueranno i valori nell'array di partenza che l'hanno fatta registrare.

Dati di partenza:

```
[2664.85 3197.32 3236.52 3209.73 3023.45 3579.94 2949.93 3401.15 3628.14 3555.29 3539.31 3536.68]
```

Array scalati di un valore pari a *periodo-1* per il calcolo degli incrementi (in questo caso periodo=3, quindi scalamento di 2 posizioni):



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel codice seguente si riportano le funzioni carica_dati(), crea_array_confronto(), calcola_incrementi() e parte del main(). Per chiarezza, in basso si riportano graficamente due esempi di valori utilizzati per calcolare le variazioni.

```
import numpy as np

FILENAME = "dati_finanziari.txt"

def carica_dati():
    return np.genfromtxt(FILENAME)

def crea_array_confronto(prezzi,periodo):
    array_confronto = prezzi[:-periodo+1]
    return array_confronto

def calcola_incrementi(dati, periodo=3):
    if periodo<=1 or periodo>=dati.size:
        raise ValueError("Errore: periodo non adeguato alla quantità di dati")
    dati2 = crea_array_confronto(dati, periodo)
    dati = dati[periodo-1:]
    variazioni = np.round(dati-dati2,2)
    return variazioni

def main():
    dati = carica_dati()
    print("DATI FINANZ =",dati)
    print("VARIAZIONI = ",calcola_incrementi(dati))

main()
```

571.67

12.41

```
DATI FINANZ = [2664.85 3197.32 3236.52 3209.73 3023.45 3579.94 2949.93 3401.15 3628.14
 3555.29 3539.31 3536.68]
VARIAZIONI =  [ 571.67   12.41 -213.07  370.21  -73.52 -178.79  678.21  154.14  -88.83
  -18.61]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Definiamo, ora, la funzione statistiche() e modifichiamo la funzione main() per includere l'invocazione a tale funzione.

Nella funzione statistica si calcola il minimo e il massimo dell'array delle variazioni. Si individuano, inoltre, le posizioni dell'array corrispondenti al minimo e al massimo (in questo caso minimo = -213,07 in posizione 2 e massimo=678,21 in posizione 6). Successivamente, si utilizzano tali posizioni per individuare a quali valori del titolo dell'array di partenza corrispondono le variazioni. Nell'esempio, il minimo (variazione pari a -213,07 posizione 2) si ottiene per il valore 3.236,52 (in posizione 2 nell'array dei valori) e il massimo (variazione pari a 678,21 posizione 6) si ottiene per 2.949,93 (in posizione 6 nell'array dei valori).

```
DATI FINANZ = [2664.85 3197.32 3236.52 3209.73 3023.45 3579.94 2949.93 3401.15 3628.14
 3555.29 3539.31 3536.68]
```

```
def statistiche(dati, variazioni):
    minimo = np.amin(variazioni)
    massimo = np.amax(variazioni)

    pos_minimo = np.where(variazioni==minimo)[0][0]
    pos_massimo = np.where(variazioni==massimo)[0][0]

    print("Maggiore variazione negativa ottenuta pari a " + str(minimo) + " sul valore " + str(dati[pos_minimo]))
    print("Maggiore variazione positiva ottenuta pari a " + str(massimo) + " sul valore " + str(dati[pos_massimo]))


def main():
    dati = carica_dati()
    try:
        variazioni = calcola_incrementi(dati,3)
        statistiche(dati, variazioni)
    except ValueError as verr:
        print(verr)

main()
```

```
Maggiore variazione negativa ottenuta pari a -213.07 sul valore 3236.52
Maggiore variazione positiva ottenuta pari a 678.21 sul valore 2949.93
```

```
VARIAZIONI =  [ 571.67   12.41 -213.07  370.21  -73.52 -178.79  678.21  154.14  -88.83
  -18.61]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev, D. (2017). Data science con Python. Apogeo.
- Maggi, G. (2020). Data science con Python. Edizioni LSWR.
- Marin, I., Shukla, A. Sarang, VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).